SC2006 Software Engineering

Lab 4 Deliverables: Testing

Lab group	SCSL, Group 4
Team	CleanCravings
Members	Rafael, Albin, Samrithi, Jovan, Jun Ren
Author(s) of this document	Rafael, Albin, Samrithi
Test run	2026-11-10, code at commit 0737ea1: 34 of 34 test cases passed

Contents

1. Black-box testing: HygieneController
    1. The control class
    2. Equivalence classes and boundary values
    3. Test cases and results
2. White-box testing: basis path testing
    1. RuleEngine.evaluate(): control flow graph, basis paths, test cases and results
    2. ClosureController.centre_status(): control flow graph, basis paths, test cases and results
3. How the tests are run

1. Black-box testing: HygieneController

1.1 The control class

HygieneController (`backend/app/controllers/hygiene_controller.py`) holds the food-hygiene logic (FR-2.1 to FR-2.5). It works out a stall's current SAFE grade from the dated snapshots imported from SFA, builds its track record, and decides whether a stall meets a user's *Hygiene Threshold*: the minimum SAFE grade the user wants to see. `passes_threshold(stall, threshold)` makes that decision for a stall using the rule in `grade_passes()`, and the centre list filter, the centre page's stall list and the recommendations all use that one rule (FR-2.5, FR-5.4), so it decides which stalls a user is shown.

The rules it implements:

- Grades are ordered **A > B > NEW > C**. NEW (a stall licensed less than a year) ranks above C, because SFA says NEW must not be presented as a failing grade (FR-2.2).
- A stall passes if its current grade ranks at or above the minimum. With no minimum set, every stall passes.
- The current grade is the newest snapshot. If the licence was in the latest SFA export but its newest grade snapshot is older than that export, SFA no longer lists a grade for it, so it has no current grade.
- A stall with no current grade, or no licensee linked, fails any minimum: the app never shows an ungraded stall to someone who asked for a minimum grade.

1.2 Equivalence classes and boundary values

Equivalence class testing splits each input's domain into classes the program should treat the same way, and tests at least one value from each class. **Boundary value testing** adds the values at and next to the edges of ordered classes, where off-by-one mistakes happen. SAFE grades are ordered and snapshot dates are ordered, so both have boundaries.

Inputs: the **minimum grade** (threshold) and the **stall's grade record** (its snapshots, the date of the latest SFA export that listed its licence, and whether a licensee is linked).

Class	Description	Valid?	Expected behaviour
EC1	No minimum set	Valid	Shown, whatever the grade
EC2	Current grade ranks at or above the minimum	Valid	Shown
EC3	Current grade ranks below the minimum	Valid	Hidden
EC4	Minimum set, stall has no SAFE grade on record	Valid	Hidden
EC5	Minimum set, no licensee linked to the stall	Valid	Hidden
EC6	Newest snapshot older than the latest export listing the licence (grade no longer listed)	Valid	Hidden
EC7	Minimum is not one of A, B, NEW, C (e.g. "D", "a")	Invalid	Rejected by the API (HTTP 422)

Boundary values. Ranks: A = 4, B = 3, NEW = 2, C = 1. For a minimum *m* the boundaries are a grade one rank below *m*, equal to *m* and one rank above *m*; the lowest (C) and highest (A) minimums are tested at their edges. For EC6 the boundary is the snapshot date: equal to the last export date (still current) versus one day older (no longer current).

Minimum	One rank below	At the minimum	One rank above
A	B: hidden (BB-11)	A: shown (BB-10)	none
B	NEW: hidden (BB-05)	B: shown (BB-03)	A: shown (BB-04)

NEW	C: hidden (BB-08)	NEW: shown (BB-07)	covered by B and A rows
C	none	C: shown (BB-09)	covered by NEW rows

1.3 Test cases and results

Method under test: `HygieneController.passes_threshold(stall, threshold)`. "Shown" means it returned `True`. Snapshot and export dates are in 2026. BB-17 and BB-18 go through the API, which validates the minimum before it reaches the controller.

No.	Covers	Test Input	Expected Output	Actual Output	Pass?
BB-01	EC1 no minimum	minimum = `None` grades (newest first): C (2026-11-01) last export: 2026-11-01	True (shown)	True (shown)	**Yes**
BB-02	EC1 no minimum, ungraded	minimum = `None` grades (newest first): none last export: 2026-11-01	True (shown)	True (shown)	**Yes**
BB-03	EC2, boundary: grade = minimum	minimum = `B` grades (newest first): B (2026-11-01) last export: 2026-11-01	True (shown)	True (shown)	**Yes**
BB-04	EC2, boundary: one rank above	minimum = `B` grades (newest first): A (2026-11-01) last export: 2026-11-01	True (shown)	True (shown)	**Yes**
BB-05	EC3, boundary: one rank below	minimum = `B` grades (newest first): NEW (2026-11-01) last export: 2026-11-01	False (hidden)	False (hidden)	**Yes**
BB-06	EC3 below minimum	minimum = `B` grades (newest first): C (2026-11-01) last export: 2026-11-01	False (hidden)	False (hidden)	**Yes**
BB-07	EC2, boundary: NEW = minimum (FR-2.2)	minimum = `NEW` grades (newest first): NEW (2026-11-01) last export: 2026-11-01	True (shown)	True (shown)	**Yes**
BB-08	EC3, boundary: C just below NEW	minimum = `NEW` grades (newest first): C (2026-11-01) last export: 2026-11-01	False (hidden)	False (hidden)	**Yes**
BB-09	EC2, boundary: lowest minimum C	minimum = `C` grades (newest first): C (2026-11-01) last export: 2026-11-01	True (shown)	True (shown)	**Yes**
BB-10	EC2, boundary: highest minimum A	minimum = `A` grades (newest first): A (2026-11-01) last export: 2026-11-01	True (shown)	True (shown)	**Yes**

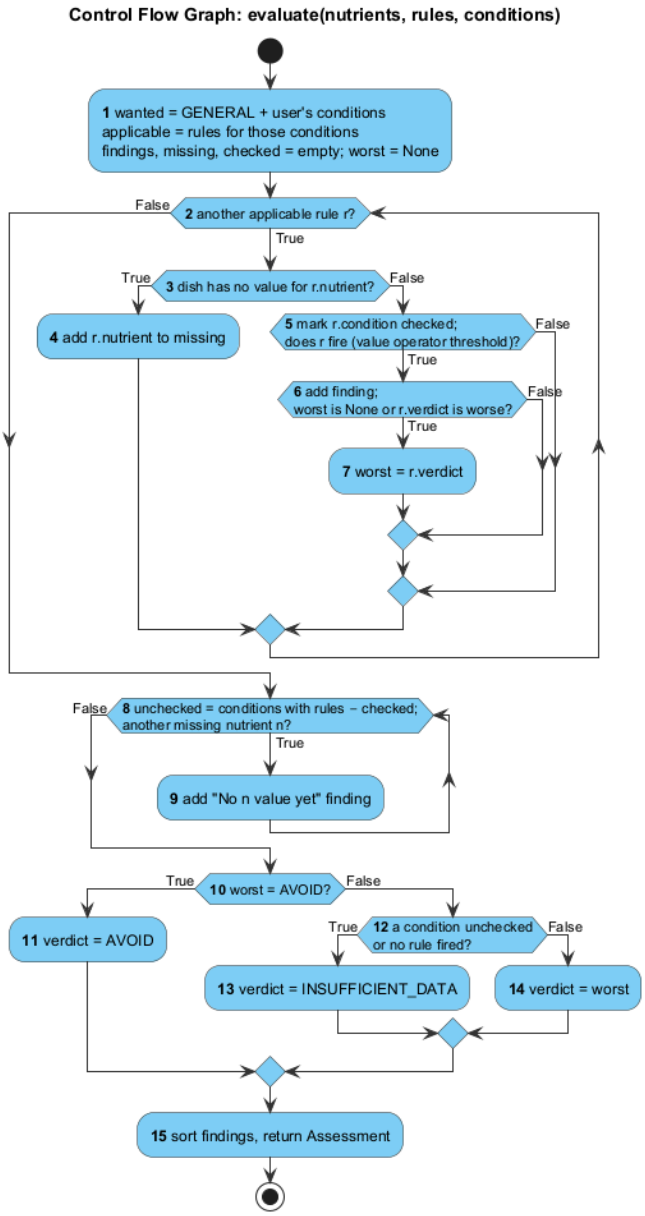
BB-11	EC3, boundary: just below A	minimum = `A` grades (newest first): B (2026-11-01) last export: 2026-11-01	False (hidden)	False (hidden)	**Yes**
BB-12	EC4 no SAFE grade on record	minimum = `C` grades (newest first): none last export: 2026-11-01	False (hidden)	False (hidden)	**Yes**
BB-13	EC5 no licensee linked	minimum = `C` grades (newest first): none no licensee	False (hidden)	False (hidden)	**Yes**
BB-14	EC2, boundary: snapshot date = last export	minimum = `A` grades (newest first): A (2026-11-01) last export: 2026-11-01	True (shown)	True (shown)	**Yes**
BB-15	EC6, boundary: snapshot 1 day older than last export	minimum = `A` grades (newest first): A (2026-10-31) last export: 2026-11-01	False (hidden)	False (hidden)	**Yes**
BB-16	EC2 newest snapshot is the current grade	minimum = `B` grades (newest first): A (2026-11-01), C (2026-05-01) last export: 2026-11-01	True (shown)	True (shown)	**Yes**
BB-17	EC7 invalid minimum: not a SAFE grade	GET /api/centres?min_grade=`D`	HTTP 422 (rejected)	HTTP 422 (rejected)	**Yes**
BB-18	EC7 invalid minimum: lower case	GET /api/centres?min_grade=`a`	HTTP 422 (rejected)	HTTP 422 (rejected)	**Yes**

2. White-box testing: basis path testing

For each method we drew the control flow graph, worked out its cyclomatic complexity *V(G)* = number of decision points + 1 (checked with *E* − *N* + 2), and chose a basis set of *V(G)* independent paths: a baseline path, then one path for each decision with that decision's outcome flipped. Each path has one test case. Node numbers in the paths match the graphs.

2.1 RuleEngine.evaluate(nutrients, rules, conditions)

`backend/app/rules/engine.py` decides a dish's verdict for a user's health conditions from the classification rules (rules as data). The worst verdict among the rules that fire wins (AVOID > MODERATE > SUITABLE). If a condition the user has could not be checked because the dish lacks that nutrient, the result is INSUFFICIENT_DATA unless a rule already said AVOID (NFR-6: never guess).



If the image is unclear, see `CFG_evaluate.png` uploaded with this document.

Cyclomatic complexity. Decision points: nodes 2, 3, 5, 6, 8, 10 and 12 (7 decisions), so *V(G)* = 7 + 1 = **8**. Check: 21 edges and 15 nodes, so 21 − 15 + 2 = 8.

Basis paths. P1 is the baseline; each later path flips one decision of the baseline:

1. Baseline: one rule, value present, rule fires, verdict MODERATE: 1-2-3-5-6-7-2-8-10-12-14-15
2. Decision 2 false at once (no applicable rules): 1-2-8-10-12-13-15
3. Decision 3 true (nutrient missing), which also makes decision 8 true: 1-2-3-4-2-8-9-8-10-12-13-15
4. Decision 5 false (rule does not fire): 1-2-3-5-2-8-10-12-13-15
5. Decision 6 false (second verdict not worse than the first): 1-2-3-5-6-7-2-3-5-6-2-8-10-12-14-15
6. Decision 10 true (a rule said AVOID): 1-2-3-5-6-7-2-8-10-11-15
7. Decision 8 true while the condition was still checked by another rule: 1-2-3-5-6-7-2-3-4-2-8-9-8-10-12-14-15
8. Decision 12 true although a rule fired (a user's condition could not be checked): 1-2-3-5-6-7-2-3-4-2-8-9-8-10-12-13-15

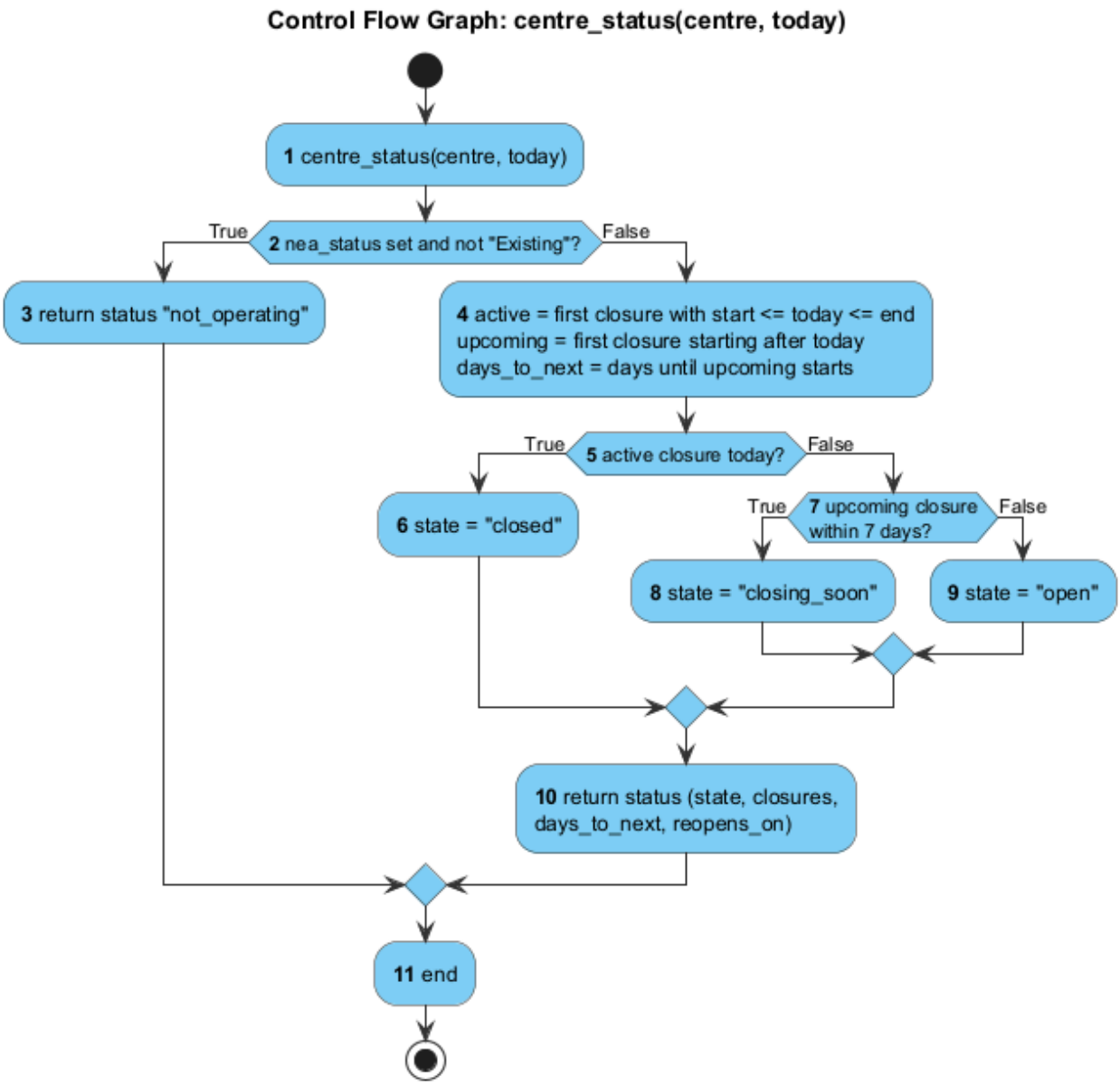
The rules in the inputs are the real rows from `classification_rules.csv`: G-SOD-01 (sodium > 2000 mg: AVOID), G-SOD-02 (sodium > 1000 mg: MODERATE), G-KCAL-01 (energy > 700 kcal: MODERATE), DM-SUG-02 (diabetes, sugar > 5 g: MODERATE).

No.	Basis path	Test Input	Expected Output	Actual Output	Pass?
WB1-1	1-2-3-5-6-7-2-8-10-12-14-15	rules: G-SOD-02 nutrients: sodium_mg = 1200 conditions: none	MODERATE	MODERATE	**Yes**
WB1-2	1-2-8-10-12-13-15	rules: none nutrients: sodium_mg = 1200 conditions: none	INSUFFICIENT_DATA	INSUFFICIENT_DATA	**Yes**
WB1-3	1-2-3-4-2-8-9-8-10-12-13-15	rules: G-SOD-02 nutrients: sodium_mg = None conditions: none	INSUFFICIENT_DATA, missing sodium_mg	INSUFFICIENT_DATA, missing sodium_mg	**Yes**
WB1-4	1-2-3-5-2-8-10-12-13-15	rules: G-SOD-02 nutrients: sodium_mg = 800 conditions: none	INSUFFICIENT_DATA	INSUFFICIENT_DATA	**Yes**
WB1-5	1-2-3-5-6-7-2-3-5-6-2-8-10-12-14-15	rules: G-SOD-02, G-KCAL-01 nutrients: sodium_mg = 1200, energy_kcal = 800 conditions: none	MODERATE	MODERATE	**Yes**
WB1-6	1-2-3-5-6-7-2-8-10-11-15	rules: G-SOD-01 nutrients: sodium_mg = 2500 conditions: none	AVOID	AVOID	**Yes**
WB1-7	1-2-3-5-6-7-2-3-4-2-8-9-8-10-12-14-15	rules: G-SOD-02, G-KCAL-01 nutrients: sodium_mg = 1200, energy_kcal = None conditions: none	MODERATE, missing energy_kcal	MODERATE, missing energy_kcal	**Yes**

WB1-8	1-2-3-5-6-7-2-3-4-2-8-9-8-10-12-13-15	rules: G-SOD-02, DM-SUG-02 nutrients: sodium_mg = 1200, sugar_g = None conditions: DIABETES	INSUFFICIENT_DATA, missing sugar_g	INSUFFICIENT_DATA, missing sugar_g	**Yes**

2.2 ClosureController.centre_status(centre, today)

`backend/app/controllers/closure_controller.py` decides whether a hawker centre is open, closing soon (a closure starts within 7 days), closed today, or no longer operating (FR-3.2). The status drives the badges on every centre card, the "closed today" banner and the exclusion of closed centres from recommendations (FR-3.3).



If the image is unclear, see `CFG_centre_status.png` uploaded with this document. Node 4 finds the first matching closures with Python's `next()`, treated as a single statement here.

Cyclomatic complexity. Decision points: nodes 2, 5 and 7 (3 decisions), so *V(G)* = 3 + 1 = **4**. Check: 13 edges and 11 nodes, so 13 − 11 + 2 = 4.

1. Baseline (open, no closures): 1-2-4-5-7-9-10-11
2. Decision 2 true (not operating): 1-2-3-11
3. Decision 5 true (closed today): 1-2-4-5-6-10-11
4. Decision 7 true (closing soon): 1-2-4-5-7-8-10-11

No.	Basis path	Test Input	Expected Output	Actual Output	Pass?

WB2-1	1-2-4-5-7-9-10-11	nea_status = Existing closures: none today = 2026-11-05	open	open	**Yes**
WB2-2	1-2-3-11	nea_status = Under Construction closures: none today = 2026-11-05	not_operating	not_operating	**Yes**
WB2-3	1-2-4-5-6-10-11	nea_status = Existing closures: 2026-11-04 to 2026-11-06 today = 2026-11-05	closed, reopens 2026-11-07	closed, reopens 2026-11-07	**Yes**
WB2-4	1-2-4-5-7-8-10-11	nea_status = Existing closures: 2026-11-08 to 2026-11-09 today = 2026-11-05	closing_soon, in 3 days	closing_soon, in 3 days	**Yes**
WB2-5	-	nea_status = Existing closures: 2026-11-12 to 2026-11-13 today = 2026-11-05	closing_soon, in 7 days	closing_soon, in 7 days	**Yes**
WB2-6	-	nea_status = Existing closures: 2026-11-13 to 2026-11-14 today = 2026-11-05	open	open	**Yes**
WB2-7	-	nea_status = Existing closures: 2026-11-01 to 2026-11-05 today = 2026-11-05	closed, reopens 2026-11-06	closed, reopens 2026-11-06	**Yes**
WB2-8	-	nea_status = Existing closures: 2026-11-01 to 2026-11-04 today = 2026-11-05	open	open	**Yes**

WB2-5 to WB2-8 are extra boundary-value cases on the same paths: a closure starting in exactly 7 days versus 8 days, and one ending today versus yesterday.

3. How the tests are run

Every test case above is an automated pytest test in `backend/tests/test_lab4.py`, run in CI on every push with the rest of the suite. This document is generated from a real run by `docs/lab4/make_test_report.py`, so the Actual Output column is what the code returned. To re-run: `cd backend`, then `pytest tests/test_lab4.py -v`.